

Blockchain Justice System

How Next.js, Node.js, TiDB, Hardhat, Hyperledger Fabric, and Docker work together to run the public case-tracking platform.

- Presenters: Lits'ito Ntsooa, Kekeletso Nalane, Mohau Nkhabe, Bokang Kobo
- Focus: problem, SMART objectives, architecture, code walkthrough, workflow, and demo

Speaker cue

This system combines a modern web interface with database storage and blockchain proof so case activity can be traced and defended.

Problem Statement

- Justice systems struggle to manage case workflows efficiently across police, investigation, commissioner, registry, DPP, and court stages.
- Traditional record systems make it hard to prove whether sensitive case data was changed after submission.
- Teams also need better visibility into who acted, when they acted, and what happened at each workflow checkpoint.

Speaker cue

The real problem is not only storage. It is trust, accountability, and being able to defend the history of each case.

System Objectives (SMART)

Next.js handles the interface, Node.js handles the logic, TiDB stores the full data, and blockchain provides proof of integrity.

- Specific: build a digital justice case management platform for capture, review, assignment, and public tracking.
- Measurable: record case actions, audit entries, workflow status, and blockchain anchors for every important event.
- Achievable: implement with Next.js, API routes, server logic, TiDB-compatible SQL storage, and blockchain anchoring.
- Relevant: improve transparency, traceability, and confidence in case handling.

Speaker cue

The objective is practical: capture the full business record in the database, then use blockchain to prove the record was not silently changed.

Architecture Overview

The repository shows a layered design with UI, API routes, server services, SQL storage, and blockchain trust services.

Speaker cue

Requests start at the interface, move through API routes and server logic, then store operational data in SQL while anchoring proof on chain.

Users

Police officers
Investigators
Commissioner
Registry and court users

Public viewers

Next.js App

Pages and dashboards
Route handlers under app/api
Form submission and navigation

Node.js Services

Authentication and sessions
Case workflow methods
Audit and business rules

Data + Trust

TiDB / MySQL tables
Hardhat contract anchor
Fabric chaincode area

Next.js - User Interface & API Layer

- Handles user interface screens such as forms, dashboards, and role-based pages.
- Exposes API routes for login, case submission, and blockchain anchoring.
- Manages user interaction, form submission, and request routing to backend logic.
- This is the layer the panel sees first during the demo.

Speaker cue

Next.js handles both the user-facing pages and the route handlers that send work into the backend.

Code to present

`buildCasePayload() -> app/police/new-case/page.tsx`

`POST() -> app/api/auth/login/route.ts`

`GET() and POST() -> app/api/cases/route.ts`

`POST() -> app/api/blockchain/anchor/route.ts`

Talk line: Next.js handles both the pages and the API routes that connect the interface to the backend.

Important line to say

Next.js handles the interface, Node.js handles the logic, TiDB stores the full data, and blockchain provides proof of integrity.

Next.js Code Snippets to Explain

- Show short code, then explain what each function does in simple words.

Speaker cue

For User 1, focus on how form data is prepared, how login works, and how case routes connect the interface to backend services.

Snippet 1: buildCasePayload()

```
const buildCasePayload = () => ({
  caseNumber: makeCaseNumber(),
  district: currentUser.station || "UNKNOWN",
  charge: allegedCrime || "UNKNOWN",
  description: modusOperandi || "N/A",
  policeSections: { sectionA: { openedByName: currentUser.name } },
})
```

Snippet 2: API routes

```
export async function POST(req: NextRequest) {
  const parsed = await readJsonBody(req)
  ...
  res.cookies.set({ name: "auth-token", value: token })
}

export async function GET(request: NextRequest) {
  const cases = await listCases()
  return NextResponse.json({ ok: true, cases })
}
```


Node.js - Server Logic & Workflow Control

- lib/server/auth-session.ts rebuilds the signed-in user from the auth cookie to protect API routes.
- lib/server/cases.ts contains the core operations: listCases(), createAndSubmitCase(), checkpointPoliceCaseStep(), and status transitions.
- The backend writes to the database inside transactions, adds audit entries, and triggers blockchain anchoring during important actions.
- This layer enforces the workflow so every stage change has business meaning and a history trail.

Speaker cue

Node.js is where the app becomes a controlled justice workflow rather than just a collection of screens.

What happens in createAndSubmitCase()

Generate a case id and case number

Normalize the payload and set status

Anchor the hash on blockchain

Insert the case row and audit entry

Node.js Code Snippets to Explain

- These are the main backend functions Kekeletso can present.

Speaker cue

This slide proves that the backend does more than receive requests. It anchors, stores, and audits each important case action.

Snippet 1: session handling

```
const sessionUser = getSessionUserFromRequest(request)
if (!sessionUser) {
  return NextResponse.json({ ok: false, error: "Unauthorized." }, { status: 401 })
}
```

This protects backend routes by confirming who is making the request.

Snippet 2: createAndSubmitCase()

```
const anchor = await anchorCaseOnChain({
  recordId: caseId,
  caseData: normalized,
  action: "CASE_CREATED_SUBMITTED",
})
await conn.query('INSERT INTO cases ...')
await conn.query('INSERT INTO audit_entries ...')
```


TiDB and Blockchain Code Snippets

- Mohau can use this page to show both storage code and smart-contract proof.
- Strong line: the database stores the full business record, while blockchain stores proof of integrity and critical action history.

Speaker cue

This makes the difference clear: the database stores the record, while the contract stores proof that the record state existed.

Snippet 1: getDbPool()

```
global.__mysqlPool = mysql.createPool({
  host: DB_HOST,
  port: Number(DB_PORT),
  user: DB_USER,
  database: DB_NAME,
  ssl,
})
```

Snippet 2: CaseAnchor.sol

```
function anchor(string calldata recordId, bytes32 contentHash, string calldata
action) external onlyWriter {
  if (latestHashForId[recordId] == contentHash) revert AlreadyAnchored();
  usedHash[contentHash] = true;
  latestHashForId[recordId] = contentHash;
  emit Anchored(recordId, contentHash, prev, msg.sender, block.timestamp,
action);
}
```

Case Workflow & Status Progression

- The workflow begins with `draft_police`, moves to `pending_investigation`, and can continue through investigation, commissioner review, DPP stages, registry intake, and court stages.
- Commissioner clarification and high-court intake methods in `lib/server/cases.ts` show that the same record keeps evolving instead of being recreated.
- Every important transition updates both the stored payload and the audit history, with an anchor attached to the new state.
- This gives the system continuity, accountability, and an inspectable chain of actions.

Speaker cue

Bokang can use this slide to explain how the record moves from office to office while keeping one trusted history.

Example statuses found in the code

`draft_police`
`pending_investigation`
`commissioner_clarification`
`approved / rejected`
`submitted_to_dpp`
`high_court_registry_intake`
`closed`

Workflow Functions Bokang Should Present

- These functions show how workflow steps are saved, locked, and moved forward.

Speaker cue

This is a strong presentation slide because it shows real workflow code, not only architecture theory.

Snippet 1: checkpointPoliceCaseStep()

```
const anchor = await anchorCaseOnChain({
  recordId: caseId,
  caseData: merged,
  action: "POLICE_STEP_CHECKPOINT",
})
await conn.query('INSERT INTO audit_entries ...')
```

Snippet 2: finalizePoliceCaseSubmission()

```
const anchor = await anchorCaseOnChain({
  recordId: input.caseId,
  caseData: merged,
  action: "CASE_SUBMITTED_TO_INVESTIGATION",
})
UPDATE cases SET status = "pending_investigation" ...
```

File Structure for Panel Questions

- This slide helps answer: where exactly is each part implemented in the repository?

Speaker cue

If the panel asks for proof, open the file listed on this slide and match it to the presenter who explained it.

Frontend

app/page.tsx
app/login/page.tsx
app/police/new-case/page.tsx
role dashboards

Backend

app/api/cases/route.ts
app/api/auth/login/route.ts
lib/server/cases.ts
lib/server/auth-session.ts

Database

lib/db.ts
lib/db/tidb.ts
db/init.sql
db/offchain.sql

Blockchain

lib/blockchain/case-anchor.ts
lib/blockchain.ts
chain/contracts/CaseAnchor.sol
fabric/chaincode/casflow/src/contract.ts

Security, Traceability & Environment

- The login route validates credentials, checks account activity, creates an auth-token cookie, and redirects users to role-specific dashboards.
- docker-compose.yml binds MySQL to 127.0.0.1 on the host, reducing accidental exposure outside the machine or Docker network.
- Checkpoint anchoring and audit entries make unauthorized changes easier to detect and investigate.
- Hardhat is used for local blockchain proof, while Fabric files in the repository show an extended enterprise blockchain direction.

Speaker cue

This slide connects implementation decisions to security and deployment concerns that panels usually ask about.

Demo guide

Suggested Demo Walkthrough

- Log in with a role account and show the role-based redirect from the login route.
- Open the police new-case form, explain the step lock/checkpoint behavior, then submit a case.
- Show that the case appears in list views and includes blockchain anchor details and audit history.
- If time allows, show a later-stage screen such as commissioner review or public tracking to prove the workflow continues beyond capture.

Speaker cue

A short live demo should follow the same story as the slides: user action, server processing, database storage, then blockchain proof.

Conclusion

Final message: full case data stays in the database, while blockchain protects trust in that data over time.

- The app is a hybrid justice platform: usable web screens, structured SQL storage, and blockchain-backed integrity checks.
- Its strength is not any one tool alone, but the way the layers support accountability together.
- The 4-user split is now clear: Lits'ito explains problem, objectives, architecture, and Next.js; Kekeletso explains backend logic; Mohau explains TiDB and blockchain; Bokang explains workflow, files, and demo.

Speaker cue

Close by repeating the architecture in one line and linking it back to transparency, accountability, and tamper detection.